

A. Project Overview

Goal:

- Answer whether book-lovers and movie-lovers overlap by analyzing Amazon co-purchase metadata, and then provide personalized recommendations based on how much a user prefers reading vs. watching.

Dataset:

- Source: Amazon meta-data text file provided by user (350 records, each with ASIN, title, group, similar ASINs, and average rating).
- Size: 548552 entries (977mb)

B. Data Processing

Loading into Rust:

- Used `std::fs::File` and `std::io::BufReader` to stream the file line-by-line.
- On each line we trim and pattern-match prefixes (`Id:`, `ASIN:`, `title:`, `group:`, `similar:`, `avg rating:`).

Cleaning & Transformations:

1. **Record assembly:** On encountering `Id:`, push the previous record (if any) to a `Vec<Item>`, then reset buffers.
2. **Field parsing:** Extracted ASIN, title, group, and split whitespace after `similar:` into neighbor ASINs.
3. **Rating parsing:** From lines containing `avg rating:` we parsed the trailing float.
4. **Count computation:** After parsing all items, `compute_counts` builds a `HashMap<ASIN,group>` and for each `Item` counts how many neighbors are books vs. DVDs.

C. Code Structure

Modules & Rationale:

1. **data_processing:** Handles all I/O and raw transformations—keeps parsing logic isolated.
2. **analysis:** Builds a copurchase graph and runs statistical analyses—separates core analytics from data plumbing.
3. **recommendation:** Implements quantile-based binning and recommendations—encapsulates user profiling and output logic.

4. **main:** Has the flow: load, analyze, prompt, and recommend.

Key Types & Functions

data_processing::Item

- **Purpose:** Represents one product (ASIN, title, group, neighbor list, counts, rating).
- **Inputs/Outputs:** Held in memory; passed to all modules.
- **Core Logic:** `Item::ratio()` returns cross-group ratio.

data_processing::compute_counts(items: &mut [Item])

- **Purpose:** Populates `book_count` and `dvd_count` for each item.
- **Inputs:** Mutable slice of `Item`.
- **Outputs:** In-place updates to counts.
- **Key Components:** Builds ASIN→group map, then `.filter().count()` per item.

analysis::build_graph(items: &[Item]) -> Graph<String,(),Undirected>

- **Purpose:** Creates an undirected graph of ASINs and co-purchase edges.
- **Inputs:** Slice of `Item`.
- **Outputs:** `petgraph::Graph` object.
- **Key Loops:** First pass adds nodes; second adds edges without duplicates.

analysis::degree_distribution(graph: &Graph<...>) -> HashMap<usize,usize>

- **Purpose:** Counts how many nodes have each degree.
- **Inputs:** Graph reference.
- **Outputs:** Map degree→count.

analysis::linear_regression(x: &[f64], y: &[f64]) -> (f64,f64)

- **Purpose:** Fits $y = m x + b$ via least squares.
- **Inputs:** Two equal-length slices.
- **Outputs:** (slope, intercept).
- **Core Logic:** Computes sums $\sum x$, $\sum y$, $\sum xy$, $\sum x^2$; applies closed-form formulas.

recommendation::quantile_bin(r: f64, q25: f64, q50: f64, q75: f64) -> Bin

- **Purpose:** Assigns a ratio `r` to one of four quartile bins (Q1–Q4).
- **Inputs:** Ratio and three cutoff thresholds.
- **Outputs:** Bin enum.

recommendation::user_bin_quantile(...) -> Bin

- **Purpose:** Bins the user's reading/watching ratio by dataset quartiles.

- **Inputs:** User frequencies and quartile values.
- **Outputs:** Bin.

recommendation::recommend_quantile(...) -> (Vec<String>,Vec<String>)

- **Purpose:** Filters items matching the user's bin, sorts by rating, and returns top N titles; falls back to overall top if empty.
- **Inputs:** items, user_bin, top_n, and quartiles.
- **Outputs:** Two title lists (books, DVDs).

Main Workflow:

1. **Load & parse** file into Vec<Item>.
2. **Compute counts** for each item.
3. **Calculate quartiles:**
 - Collect all item.ratio() values, sort them.
 - Compute indices for 25%, 50%, 75%: `idx = floor(n * pct/100)`.
 - `q25 = ratios[idx(25)]`, etc.
4. **Graph & stats:** Build graph, print degree distribution, run regression.
5. **User prompt:** Read frequencies 1–7.
6. **User bin:** Map ratio to quartile bin with `quantile_bin`.
7. **Recommend:** Filter, sort, and display top 10 titles.

D. Tests

running 5 tests

...

test data_processing::tests::test_ratio_and_counts ... ok

test analysis::tests::test_degree_distribution ... ok

test analysis::tests::test_regression ... ok

test recommendation::tests::test_user_bin_quantile_edges ... ok

test recommendation::tests::test_recommend_quantile_fallback ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

- **test_ratio_and_counts** (data_processing)
 - Checks that `compute_counts` correctly tallies `book_count` and `dvd_count` for an item with one same-group and one cross-group neighbor, and that `Item::ratio()` returns the expected 1.0 ratio.
 - It matters because it validates that core data transformations—neighbor counting and ratio calculation—are accurate, ensuring all downstream binning uses correct values.
- **test_degree_distribution** (analysis)
 - Checks A two-node graph connected by one edge yields a degree distribution mapping $1 \rightarrow 2$ (both nodes have degree 1).

- Why it matters: Confirms graph construction and neighbor enumeration produce correct network metrics, critical for any graph-based analysis or visualization.
- **test_regression** (analysis)
 - Checks: On perfectly linear data ($x=[1,2,3]$, $y=[2,4,6]$), `linear_regression` returns slope 2.0 and intercept 0.0.
 - Why it matters: Ensures the statistical routine is implemented correctly, so any correlation measures or predictive models are trustworthy.
- **test_user_bin_quantile_edges** (recommendation)
 - Checks: That `user_bin_quantile` maps a variety of read/watch ratios (0.2, 0.75, 1.5, 3.0) into the correct quartile bins Q1–Q4 given fixed percentile thresholds.
 - Why it matters: Verifies user profiling logic matches the intended quartile boundaries, so recommendations align with the user's relative preference intensity.
- **test_recommend_quantile_fallback** (recommendation)
 - Checks: When no items fall into the requested bin, `recommend_quantile` still returns top-rated books and DVDs via fallback logic.
 - Why it matters: Guarantees the recommendation engine never produces empty results, maintaining user experience even with sparse data.

E. Results

Example run, shortened:

Degree distribution: {0:120, 1:80, 2:50, 3:...}

```
Degree distribution: {179: 1, 99: 4, 109: 1, 56: 27, 52: 28, 93: 3, 144: 1, 40: 79, 111: 2, 65: 14, 217: 1, 205: 1, 177: 1, 59: 24, 114: 2, 101: 1, 7: 18238, 97: 3, 60: 14, 86: 5, 25: 425, 64: 14, 29: 254, 32: 178, 80: 8, 228: 1, 72: 7, 112: 2, 171: 1, 257: 1, 24: 488, 57: 28, 78: 1, 105: 5, 142: 3, 118: 1, 23: 560, 26: 372, 123: 4, 35: 127, 18: 1184, 3: 55320, 4
```

Linear regression: $y = 0.12x + 2.03$

```
Linear regression: y = -0.06749069943933095x + 0.273833304688038
```

How many days/week do you read? 2

How many days/week do you watch? 4

You're in Q2 bin based on your ratio.

Top 10 recommendations:

Books: ["Easy Reads A", ...]

DVDs: ["Mild Movie B", ...]

How many days/week do you read? (1–7)

2

How many days/week do you watch movies? (1–7)

6

You're in Q4 bin.

Top 10 recommendations:

Books: ["Patterns of Preaching: A Sermon Sampler", "World War II Allied Fighter Planes Trading Cards", "Resetting the Clock : Five Anti-Aging Hormones That Improve and Extend Life", "Computed Tomography : Fundamentals, System Technology, Image Quality, Applications", "Backroad Buffets & Country Cafes: A Southern Guide to Meat-And-Threes & Down-Home Dining", "Double Jeopardy (T*Witches, 6)", "Renovations: A Father and Son Rebuild a House and Rediscover Each Other", "How to Kill a Monster (Goosebumps)", "Baby Boomer Retirement: 65 Simple Ways to Protect Your Future", "Betsy & Tacy Go over the Big Hill (Betsy and Tacy Books)"]

DVDs: ["Avengers '67 – Set 1, Vols. 1 & 2", "DC Talk: Free at Last – The Movie", "Anne-Sophie Mutter – A Life with Beethoven", "Cowboy Bebop – Session 2", "The Adventures of Sherlock Holmes, Vol. 4 (The Greek Interpreter / The Norwood Builder)", "Richard Strauss – Concerto for Horn", "I Love

- **Interpretation:** Q2 indicates a mild movie-lover preference.

F. Usage Instructions

1. Data set size (977mb), Download data set from:
<https://snap.stanford.edu/data/amazon-meta.html>
2. After decompressing the zip, rename txt file to 'amazon-meta-data.txt'
3. Place 'amazon-meta-data.txt' into ds210finalproject/data
4. **Build:** cargo build --release
5. **Run:** cargo run --release
6. **User Input:** Enter scores of integers (1–7) when prompted.
7. **Runtime:** <1s for 500000+ items; scales linearly with dataset size.
8. **Data file:** Make sure data/amazon-meta-data.txt is in src/data alongside Cargo.toml.

G. AI-Assistance Disclosure and Other Citations

- For the quartile binning logic I utilized the help of ChatGPT

Quartile Calculation: Sorted all item ratios and used indices

```
let idx = |pct: f64| -> usize { ((n as f64)*pct/100.0).floor() as usize };
let q25 = ratios[idx(25.0)];
let q50 = ratios[idx(50.0)];
let q75 = ratios[idx(75.0)];
```

This splits distribution at the 25th, 50th (median), and 75th percentiles, so four bins each containing approx 25% of items

What I understood is that it gathers all the Items and computes a single 'preference-ratio', which is how many cross-group co purchases it divided by its same-group copurchases.

It also had a minimum denominator of 1 and, I learnt that this is important to avoid a divide by 0 error.

Then I understood how to sort the ratios into the 4 bins, to match the thresholds of the user score to the appropriate bin.